

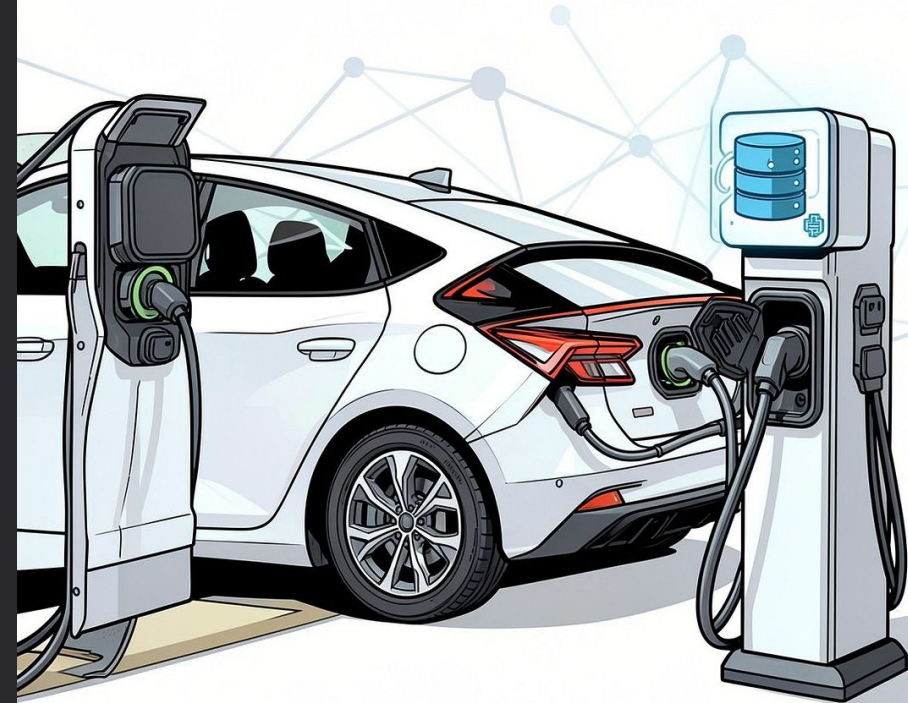
EV Charging Network Management System

TAE-2 Database Implementation & Advanced SQL

TY B.Tech – Data Science · G H Rasoni College of Engineering and Management, Pune

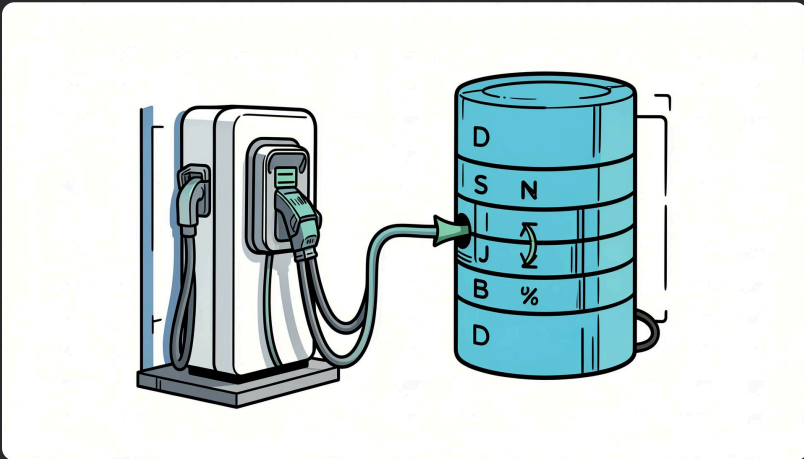
RDBMS: MySQL

DRIVING THE EMOTION
THE ELECTRIC VEHICLE



1. Domain & TAE-1 Recall

Electric Vehicle (EV) Charging Infrastructure



- Manages EV charging operators, stations, customers, vehicles, staff, sessions & payments.
- TAE-1: conceptual design → ER model & relational schema (approved).
- TAE-2: implement schema in MySQL; demonstrate advanced SQL & tuning.

Data flow (simplified):

EV Charging Infrastructure ↓ Operators ↓ Charging Stations ↓ Charging Sessions ↓ Payments

2. Database Architecture

Seven Core Tables and Their Relationships

PK / FK labels shown on diagram. Relationships:

- OPERATOR 1 : M CHARGING_STATION
- CUSTOMER 1 : M VEHICLE
- CHARGING_STATION 1 : M CHARGING_SESSION
- VEHICLE 1 : M CHARGING_SESSION
- STAFF 1 : M CHARGING_SESSION
- CHARGING_SESSION 1 : 1 PAYMENT



Only the specified tables are included; no additional entities introduced.

3. Database Instantiation

DDL, Constraints and Data Population

1

DDL

- CREATE DATABASE ev_network;
- CREATE TABLE ... (PRIMARY KEY ...)
- FOREIGN KEY (...) REFERENCES ...

2

CONSTRAINTS

- NOT NULL, DEFAULT
- CHECK (where applicable)
- ON DELETE CASCADE, ON UPDATE CASCADE

Constraints maintain referential integrity and valid values.

DATA POPULATION

Normalized insertion counts (implemented in MySQL):

OPERATOR	8
CUSTOMER	5,808
STAFF	5,235
CHARGING_STATION	5,388
VEHICLE	3,951
CHARGING_SESSION	7,000
PAYMENT	7,000

4. Complex SQL Queries

Business Analysis Using Joins, Aggregation and Subqueries



Station Performance

INNER JOIN + GROUP BY → total sessions, total energy, total revenue, average amount



High-Usage Stations

GROUP BY + HAVING (COUNT(Session_ID) > 20)



Customer Charging Analysis

CUSTOMER → VEHICLE → CHARGING_SESSION: total sessions, energy, spending

4. Complex SQL Queries

Business Analysis Using Joins, Aggregation and Subqueries



Operator Performance

OPERATOR → CHARGING_STATION → CHARGING_SESSION



Self Join

CHARGING_STATION self-join to find multiple stations for same Operator_Name

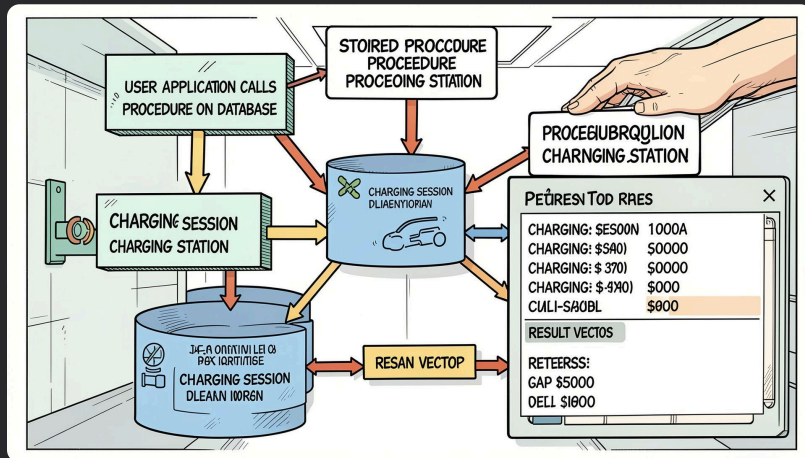


Correlated Subquery

Find sessions where energy > station average (correlated per station)

5. Stored Procedure

Parameterized Operational Transaction



Procedure: GetStationSessions(p_Station_ID VARCHAR(20))

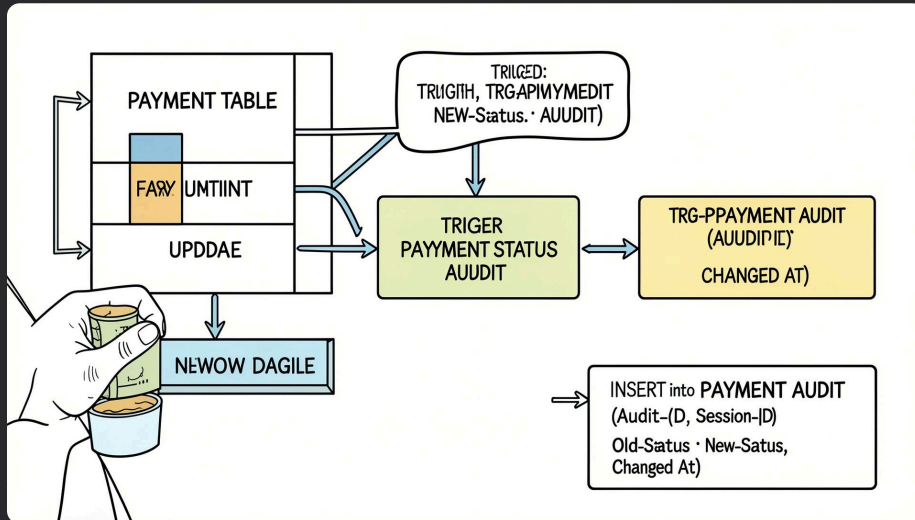
- Purpose: Retrieve all charging sessions for a selected station.
- Returns: Session_ID, Station_ID, Station_Name, City, Vehicle_No, Staff_ID, Connector_Type, Start_Date, Start_Time, End_Time, Energy, Amount, Issue_Type.

```
CALL GetStationSessions('ST1001');
```

- ❏ Parameterized procedures enable reusable, secure operations without rewriting queries.

6. Trigger & Dynamic Auditing

Automatically Recording Payment Status Changes



Trigger: trg_payment_status_audit — AFTER UPDATE ON PAYMENT

Logic:

1. On UPDATE, compare OLD.Payment_Status and NEW.Payment_Status
2. If changed → INSERT audit record into PAYMENT_AUDIT

PAYMENT_AUDIT columns: Audit_ID, Session_ID, Old_Status, New_Status, Changed_At

```
IF OLD.Payment_Status <> NEW.Payment_Status THEN INSERT  
INTO PAYMENT_AUDIT (...); END IF;
```

7. Business Reporting Views

Two Virtual Views for Business Reporting

Station_Performance (View)

Purpose: Station-level reporting

- Station_ID, Station_Name, Operator, State, City, Station_Status
- Total_Sessions, Total_Energy, Total_Revenue, Avg_Revenue_per_Session

Source: CHARGING_STATION + CHARGING_SESSION

Customer_Charging_Summary (View)

Purpose: Customer-level analysis

- Customer_ID, Customer_Name, Phone, Email
- Total_Sessions, Total_Energy, Total_Amount, Avg_Amount

Source: CUSTOMER ↓ VEHICLE ↓ CHARGING_SESSION

Create with: CREATE OR REPLACE VIEW ... (virtual MySQL views — not materialized)

8. Performance Tuning & Indexing

EXPLAIN Before and After Indexing

Test 1 — Station + Date

Query filters: Station_ID, Start_Date

Index (composite):

```
CREATE INDEX idx_session_station_date ON  
CHARGING_SESSION (Station_ID, Start_Date);
```

Flow: EXPLAIN BEFORE → CREATE COMPOSITE INDEX →
EXPLAIN AFTER

Test 2 — Vehicle

Query filter: Vehicle_No

Index (single-column):

```
CREATE INDEX idx_session_vehicle ON CHARGING_SESSION  
(Vehicle_No);
```

Flow: EXPLAIN BEFORE → CREATE INDEX → EXPLAIN AFTER

What we compare with EXPLAIN: type, key, rows, Extra